Some Useful Python Functions

Contents

1 Typecasting **2**
1.1 float() . . . 2
1.2 int() . . . 2

2 NumPy Functions **3**
2.1 np.eye . . . 3
2.2 np.zeros . . . 3
2.3 np.ones_like . . . 4
2.4 np.diag . . . 5
2.5 np.append . . . 5
2.6 np.vstack . . . 6
2.7 np.sum . . . 6
2.8 np.average . . . 7
2.9 np.argmin . . . 8
2.10 np.where . . . 8
2.11 np.count_nonzero . . . 9
2.12 np.arange . . . 10
2.13 np.clip . . . 10
2.14 np.linalg.eig . . . 11
2.15 np.linalg.lstsq . . . 12
2.16 np.linalg.svd . . . 13
2.17 np.linalg.matrix_power . . . 14
2.18 np.linalg.norm . . . 14
2.19 np.linalg.solve . . . 15
2.20 np.linalg.inv . . . 16

3 Generating Random Numbers **17**
3.1 random.randint . . . 17
3.2 random.random . . . 17
3.3 random.sample . . . 18

4 Reading Files and IO **19**
4.1 np.loadtxt . . . 19
4.2 pd.read_csv . . . 20

5 Other **21**
5.1 scipy.linalg.solve_triangular . . . 21
5.2 scipy.linalg.lu . . . 22
5.3 math.ceil . . . 23

1 Typecasting

1.1 float()

Description: The `float()` function converts a number or a string to a floating-point number.

Syntax:

```
float([x])
```

Inputs:

`x` : A number or a string that represents a number.

Output:

- (float) A floating-point number.

Usage Example:

```
# Convert an integer to a float
x = float(42)
print(x)

# Convert a string to a float
y = float("3.14")
print(y)
```

1.2 int()

Description: The `int()` function converts a number or a string to an integer.

Syntax:

```
int(x, base=10)
```

Inputs:

`x` : A number or a string that represents a number.

`base` : (optional) The base in which the string is if `x` is a string. Default is 10.

Output:

- (int) An integer.

Usage Example:

```
# Convert a float to an integer
x = int(3.14)
print(x)

# Convert a string to an integer in base 2
y = int("1101", 2)
print(y)
```

2 NumPy Functions

Documentation provided for latest version of `numpy`. You can check the version you're on using `numpy.version.version`. Many functions listed here used to be a part of `scipy` library.

2.1 np.eye

Description: The `numpy.eye` function creates an identity matrix, which is a square matrix with ones on the main diagonal and zeros elsewhere.

Syntax:

```
np.eye(N, M=None, k=0, dtype=<class 'float'>, order='C')
```

Inputs:

- `N` (int): The number of rows.
- `M` (int, optional): The number of columns. If not specified, M is set to N.
- `k` (int, optional): The diagonal offset. Default is 0, which represents the main diagonal.
- `dtype` (data-type, optional): The data type of the output array.
- `order` ('C', 'F', optional): The order of the array. Default is 'C'.

Output:

- (ndarray): A 2-D array with ones on the diagonal and zeros elsewhere.

Usage Example:

```
import numpy as np

# Create a 3x3 identity matrix
identity_matrix = np.eye(3)
print(identity_matrix)
```

2.2 np.zeros

Description: The `numpy.zeros` function creates an array filled with zeros.

Syntax:

```
np.zeros(shape, dtype=float, order='C')
```

Inputs:

- `shape` (int or tuple of ints): The shape of the array.
- `dtype` (data-type, optional): The data type of the output array. Default is float.

`order` ('C', 'F', optional): The order of the array. Default is 'C'.

Output:

- (ndarray): A new array of the given shape and data type, filled with zeros.

Usage Example:

```
import numpy as np

# Create a 2x4 array filled with zeros
zero_array = np.zeros((2, 4))
print(zero_array)
```

2.3 np.ones_like

Description: The `np.ones_like` function returns an array of ones with the same shape and type as a given array.

Syntax:

```
        np.ones\_like(a, dtype=None, order='K', shape=None)
```

Inputs:

`a` (array-like): The shape and data-type of `a` define these same attributes of the returned array.

`dtype` (data-type, optional): Overrides the data type of the output array.

`order` ('C', 'F', 'A' or 'K', optional): The order of the array. Default is 'K'.

`shape` (int or sequence of ints, optional): Overrides the shape of the result.

Output:

- (ndarray): A new array of the given shape and data type, filled with zeros.

Usage Example:

```
        import numpy as np

        # Create a 2x4 array filled with zeros
        zero_array = np.zeros((2, 4))
        print(zero_array)
```

2.4 np.diag

Description: The `np.diag` function extracts or constructs a diagonal array.

Syntax:

```
np.diag(v, k=0)
```

Inputs:

- `v` (array-like): If `v` is a 2-D array, it returns the diagonal of `v` with the given offset.
- `k` (int, optional): The diagonal in question. The default is 0, which refers to the main diagonal.

Output:

- (ndarray): The extracted or constructed diagonal array.

Usage Example:

```
import numpy as np

# Create a diagonal matrix with specified values
values = [1, 2, 3]
diagonal = np.diag(values)
print(diagonal)
```

2.5 np.append

Description: The `np.append` function appends values to the end of an array.

Syntax:

```
np.append(arr, values, axis=None)
```

Inputs:

- `arr` (array-like): Input array.
- `values` (array-like): Values to append.
- `axis` (int, optional): The axis along which `values` are appended.

Output:

- (ndarray): A new array with `values` appended.

Usage Example:

```
import numpy as np

# Define an input array 'arr' and 'values' to append
arr = np.array([1, 2, 3])
values = np.array([4, 5, 6])

# Append 'values' to 'arr'
result = np.append(arr, values)
print(result)
```

2.6 np.vstack

Description: The `np.vstack` function stacks arrays in a vertical sequence.

Syntax:

```
np.vstack(tup)
```

Inputs:

`tup` (sequence of ndarrays): The arrays to stack vertically.

Output:

- (ndarray): The stacked array.

Usage Example:

```
import numpy as np

# Create two arrays
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])

# Stack them vertically
stacked_array = np.vstack((array1, array2))
print(stacked_array)
```

2.7 np.sum

Description: The `np.sum` function computes the sum of array elements over a specified axis.

Syntax:

```
np.sum(a, axis=None, dtype=None, out=None, keepdims=False, initial
    =0)
```

Inputs:

`a` (array-like): Input data.

`axis` (int or tuple of ints, optional): The axis or axes over which the sum is computed.

`dtype` (data-type, optional): The data-type of the returned array.

`out` (ndarray, optional): An output array in which to place the result.

`keepdims` (bool, optional): If True, the result will broadcast correctly against the original array.

`initial` (scalar, optional): The starting value for the sum.

Output:

- (ndarray): The sum of array elements.

Usage Example:

```
import numpy as np

# Calculate the sum of an array
arr = np.array([1, 2, 3, 4, 5])
result = np.sum(arr)
print(result)
```

2.8 np.average

Description: The `np.average` function computes the weighted average along a specified axis.

Syntax:

```
np.average(a, axis=None, weights=None)
```

Inputs:

`a` (array-like): Input data.

`axis` (int, optional): The axis along which to compute the average.

`weights` (array-like, optional): An array of weights to apply. The array of weights must be the same shape as a if no axis is specified, otherwise the weights must have dimensions and shape consistent with a along the specified axis. The calculation is:

`avg = sum(a * weights) / sum(weights)`

Output:

- (ndarray): The weighted average of the input data.

Usage Example:

```
import numpy as np

# Compute the weighted average of an array
data = np.array([1, 2, 3, 4])
weights = np.array([0.2, 0.3, 0.1, 0.4])
average = np.average(data, weights=weights)
print(average)
```

2.9 np.argmin

Description: The `np.argmin` function returns the indices of the minimum values along an axis. Note that `np.argmax` works similarly.

Syntax:

```
np.argmin(a, axis=None, out=None, keepdims=False)
```

Inputs:

- `a` (array-like): Input array.
- `axis` (int, optional): The axis along which to operate.
- `out` (ndarray, optional): An output array in which to place the result.
- `keepdims` (bool, optional): If this is set to True, the result will broadcast correctly against the array.

Output:

- (ndarray or int): The indices of the minimum values.

Usage Example:

```
import numpy as np

# Find the indices of minimum values in an array
arr = np.array([3, 1, 5, 2, 4])
indices = np.argmin(arr)
print(indices)
```

2.10 np.where

Description: The `np.where` function works differently based on if `x` and `y` are specified or not. If specified, returns elements chosen from `x` or `y` depending on `condition`. If not, returns indices (as ndarray) where the condition is `True`.

Syntax:

```
np.where(condition, [x, y])
```

Inputs:

`condition` (array-like, bool): The condition to be checked element-wise.

`x` (array-like, optional): Values to use where the condition is True.

`y` (array-like, optional): Values to use where the condition is False.

Note: If `x` is specified, `y` must be too.

Output:

- (ndarray): An array with elements from `x` where condition is True, and elements from `y` elsewhere.

Usage Example:

```
import numpy as np

# Create a 2x3 random array
arr = np.random.randn(2,3)

# Find the indices of elements greater than 0
indices = np.where(arr > 0)
print(indices)

# Replace entries of arr so it has no negative entries
non_neg = np.where(arr > 0, arr, 0)
print(non_neg)
```

2.11 np.count_nonzero

Description: The `np.clip` function counts the number of non-zero values in an array.

Syntax:

```
        np.clip(a, axis= None, keepdims = False)
```

Inputs:

`a` (array-like): Input data.

`axis` (int or tuple, optional): Axis or tuple of axes along which to count non-zeros. Default is 'None', meaning that non-zeros will be counted along a flattened version of `a`.

`keepdims` (bool, optional): If this is set to True, the axes that are counted are left in the result as dimensions with size one.

Output:

- (int or array of int): Number of non-zero values in the array along a given axis. Otherwise, the total number of non-zero values in the array is returned.

Usage Example:

```
import numpy as np

# Number of non-zero entries in a diagonal matrix
print(np.count_nonzero(np.eye(4)))

# Number of non-zero entries along various axes
a = np.array([[0, 1, 7, 0], [3, 0, 2, 19]])

print(np.count_nonzero(a))
print(np.count_nonzero(a, axis=0))
print(np.count_nonzero(a, axis=1))
print(np.count_nonzero(a, axis=1, keepdims=True))
```

2.12 np.arange

Description: The `np.arange` function creates an array with regularly spaced values within a specified range.

Syntax:

```
np.arange([start,] stop[, step,])
```

Inputs:

`start` (optional): The starting value of the sequence (default is 0).

`stop` (required): The end value of the sequence.

`step` (optional): The step size between values (default is 1).

Output:

- (ndarray): An array containing values in the specified range.

Usage Example:

```
import numpy as np

# Create an array with values from 0 to 9 (exclusive)
result = np.arange(10)
print(result)
```

2.13 np.clip

Description: The `np.clip` function limits the values in an array to be within a specified range.

Syntax:

```
np.clip(a, a_min, a_max, out=None)
```

Inputs:

- `a` (array-like): Input data.
- `a_min` (scalar or array-like): Minimum value.
- `a_max` (scalar or array-like): Maximum value.
- `out` (ndarray, optional): An output array in which to place the result.

Output:

- (ndarray): An array with values clipped to the specified range.

Usage Example:

```
import numpy as np

# Clip values in an array to the range [1, 5]
arr = np.array([0, 2, 6, 3, 7])
clipped = np.clip(arr, 1, 5)
print(clipped)
```

2.14 np.linalg.eig

Description: The `np.linalg.eig` function computes the eigenvalues and right eigenvectors of a square matrix.

Syntax:

```
np.linalg.eig(a)
```

Inputs:

- `a` (ndarray): The square matrix for which to compute the eigenvalues and eigenvectors.

Output:

- (tuple) A tuple of two arrays: eigenvalues; each repeated according to its multiplicity (not necessarily ordered); and right eigenvectors; normalized and such that the column `eigenvectors[:,i]` is the eigenvector corresponding to the eigenvalue `eigenvalues[i]`.

Usage Example:

```
import numpy as np
import np.linalg as la

# Define a square matrix 'a'
```

```
a = np.array([[1, 2],
              [3, 4]])

# Compute the eigenvalues and right eigenvectors
eigenvalues, eigenvectors = la.eig(a)
print("Eigenvalues:")
print(eigenvalues)
print("Eigenvectors:")
print(eigenvectors)
```

2.15 np.linalg.lstsq

Description: The `np.linalg.lstsq` function returns the least-squares solution to a linear matrix equation. Which is, computes the vector `x` that approximately solves the equation `a @ x = b`, minimizing ℓ-2 norm $||b - ax||$.

Syntax:

```
np.linalg.lstsq(a, b, rcond=None)
```

Inputs:

- `a` (ndarray): The coefficient matrix.
- `b` (ndarray): Right-hand side.
- `rcond` (float, optional): Cutoff for 'small' singular values; used to determine effective rank.

Output:

- (tuple) A tuple of four arrays: solution, residuals (sum of squared residuals), effective rank, singular values.

Usage Example:

```
import numpy as np
import np.linalg as la

# Define a coefficient matrix 'a' and a right-hand side 'b'
a = np.array([[1, 2],
              [3, 4],
              [5, 6]])
b = np.array([7, 8, 9])

# Solve the linear least-squares problem
solution, residuals, eff_rank, singular_values = la.lstsq(a, b)
print("Solution:")
print(solution)
print("Residuals:")
print(residuals)
```

```
print("Effective Rank:")
print(eff_rank)
print("Singular Values:")
print(singular_values)
```

2.16 np.linalg.svd

Description: The `np.linalg.svd` function performs Singular Value Decomposition on a matrix.

Syntax:

```
np.linalg.svd(a, full_matrices=True, compute_uv=True, hermitian=
    False)
```

Inputs:

- `a` (ndarray): The input matrix.
- `full_matrices` (bool, optional): If True, compute full-sized U and Vh.
- `compute_uv` (bool, optional): If True, compute U and Vh.
- `hermitian` (bool, optional): If True, 'a' is assumed to be Hermitian (conjugate symmetric).

Output:

- (U, s, Vh) (ndarray, ndarray, ndarray): U, s, and Vh such that a == U @ np.diag(s) @ Vh.

Usage Example:

```
import numpy as np
import np.linalg as la

# Define an input matrix 'a'
a = np.array([[1, 2],
              [3, 4],
              [5, 6]])

# Perform Singular Value Decomposition
U, s, Vh = la.svd(a)
print("U:")
print(U)
print("s:")
print(s)
print("Vh:")
print(Vh)
```

2.17 np.linalg.matrix_power

Description: The `np.linalg.matrix_power` function raises a square matrix to the specified integer power.

Syntax:

```
np.linalg.matrix_power(a, n)
```

Inputs:

- `a` (ndarray): The square matrix to be raised to a power.
- `n` (int): The integer power to which the matrix is raised.

Output:

- (ndarray) The result of raising the matrix to the specified power.

Usage Example:

```
import numpy as np
import np.linalg as la

# Define a square matrix 'a'
a = np.array([[1, 2],
              [3, 4]])

# Raise the matrix to the power of 3
result = la.matrix_power(a, 3)
print(result)
```

2.18 np.linalg.norm

Description: The `np.linalg.norm` function calculates the matrix or vector norm. Norm type depends on `ord`.

Syntax:

```
np.linalg.norm(x, ord=None, axis=None, keepdims=False)
```

Inputs:

- `x` (array-like): Input array or matrix.
- `ord` (None, 'fro', 'nuc', float, int, inf, -inf, ... , optional): Order of the norm.
- `axis` (None, int, 2-tuple of ints, optional): The axis or axes along which the norm is computed.
- `keepdims` (bool, optional): If this is set to True, the axes which are normed are left in the result as dimensions with size one.

Output:

- (float or ndarray) The computed matrix or vector norm.

Usage Example:

```
import numpy as np
import np.linalg as la


# Define a vector 'x'
x = np.array([1, 2, 3])

# Calculate the L2 norm of the vector
l2_norm = la.norm(x, ord=2)
print(l2_norm)
```

2.19 np.linalg.solve

Description: The `np.linalg.solve` function solves a linear matrix equation. Which is, computes the "exact" solution x for matrix equation $ax = b$.

Syntax:

```
np.linalg.solve(a, b)
```

Inputs:

- `a` (ndarray): The coefficient matrix.
- `b` (ndarray): Right-hand side.

Output:

- (ndarray) The solution to the linear matrix equation.

Usage Example:

```
import numpy as np
import np.linalg as la

# Define a coefficient matrix 'a' and a right-hand side 'b'
a = np.array([[1, 2],
              [3, 4]])
b = np.array([5, 6])

# Solve the linear matrix equation
solution = la.solve(a, b)
print(solution)
```

2.20 np.linalg.inv

Description: The `np.linalg.inv` function computes the multiplicative inverse of a matrix.

Syntax:

```
np.linalg.inv(a)
```

Inputs:

- `a` (array-like): The input matrix to invert.

Output:

- (ndarray): The inverse of the input matrix.

Usage Example:

```
import numpy as np
import np.linalg as la

# Create a matrix
matrix = np.array([[1, 2], [3, 4]])

# Calculate the inverse of the matrix
inverse_matrix = la.inv(matrix)
print(inverse_matrix)
```

3 Generating Random Numbers

Listed are some random number generation functions that you might useful. Please refer to Python's random number generation webpage for additional functions. Note that `numpy` library has similar random functions. Refer to Numpy's legacy random number generation webpage for more information.

3.1 random.randint

Description: The `random.randint` function generates a random integer between specified values.

Syntax:

```
random.randint(a, b)
```

Inputs:

- `a` (int): The lower bound for the random integer.
- `b` (int): The upper bound for the random integer.

Output:

- (int): A random integer between `a` and `b` (inclusive).

Usage Example:

```
import random

# Generate a random integer between 1 and 10
random_int = random.randint(1, 10)
print(random_int)
```

3.2 random.random

Description: The `random.random` function generates a random floating-point number.

Syntax:

```
        random.random()
```

Output:

- (float): A random floating-point number between 0.0 (inclusive) and 1.0 (exclusive).

Usage Example:

```
        import random

        # Generate a random integer between 1 and 10
        random_float = random.random()
        print(random_float)
```

3.3 random.sample

Description: The `random.sample` function returns a list with a random selection of unique elements from a given sequence without replacement.

Syntax:

```
random.sample(population, k)
```

Inputs:

`population` (sequence): The population to sample from.

`k` (int): The number of unique elements to sample.

Output:

- (list): A list of `k` unique elements randomly sampled from the population.

Usage Example:

```
import random

# Sample 3 unique elements from a list
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
sampled_list = random.sample(my_list, 3)
print(sampled_list)
```

4 Reading Files and IO

4.1 np.loadtxt

Description: The `np.loadtxt` function loads data from a text file.

Syntax:

```
np.loadtxt(fname, dtype=<class 'float'>, comments='#', delimiter=
    None, converters=None, skiprows=0, usecols=None, unpack=False,
    ndmin=0, encoding=None, max_rows=None, quotechar=None)
```

Inputs:

`fname` (file, str, or pathlib.Path): The file, filename, or generator to read.

`dtype` (data-type, optional): The data-type of the resulting array.

`comments` (str or sequence of str, optional): The characters that mark the start of a comment.

`delimiter` (str, optional): The string used to separate values.

`converters` (dict, optional): A dictionary mapping column numbers to functions.

`skiprows` (int, optional): The number of lines to skip at the beginning of the file.

`usecols` (sequence, optional): Which columns to read, with 0 being the first.

`unpack` (bool, optional): If True, the returned array is transposed.

`ndmin` (int, optional): The number of dimensions the resulting array should have.

`encoding` (str, optional): The encoding used to decode the input file. Default is 'None', special value 'bytes' enables backward compatibility workarounds.

`max_rows` (int, optional): The max number of rows read.item[`quotechar`] (unicode character or None, optional): The character used to denote the start and end of a quoted item.

Output:

- (ndarray): The loaded data as an array.

Usage Example:

```
import numpy as np

# Load data from a text file
data = np.loadtxt('data.txt', delimiter=',')
print(data)
```

4.2 pd.read_csv

Description: The `pd.read_csv` function reads a CSV (Comma-Separated Values) file and returns a DataFrame.

Syntax:

```
pd.read_csv(filepath_or_buffer, sep=',', delimiter=None, header='
    infer', names=None, index_col=None, usecols=None, dtype=None,
    encoding='utf-8')
```

Inputs:

`filepath_or_buffer` (str or file-like): The file path or a file-like object.

`sep` (str, default=','): The delimiter used to separate fields.

`delimiter` (str, default=None): An alternative delimiter.

`header` (int, list of int, default='infer'): Row(s) to use as the column names.

`names` (array-like, default=None): Names to use for the columns.

`index_col` (int or str, default=None): Column(s) to set as index (row labels).

`usecols` (list-like or callable, default=None): Columns to read.

`dtype` (Type name or dict of column-¿type, default=None): Data type to force.

`encoding` (str, default='utf-8'): Encoding to use for UTF when reading files.

Output:

- (DataFrame): A table-like data structure.

Usage Example:

```
import pandas as pd

# Read a CSV file into a DataFrame
data = pd.read_csv('data.csv')
print(data)
```

5 Other

5.1 scipy.linalg.solve_triangular

Description: The `scipy.linalg.solve_triangular` function solves a triangular system of linear equations.

Syntax:

```
scipy.linalg.solve_triangular(a, b, trans=0, lower=False,
   unit_diagonal=False, overwrite_b=False, check_finite=True
   )
```

Inputs:

- `a` (array-like): The coefficient matrix.
- `b` (array-like): The right-hand side of the equation.
- `trans` (int, optional): Transposition of the coefficient matrix. Default is 0.
- `lower` (bool, optional): Whether the matrix is lower or upper triangular. Default is False (upper triangular).
- `unit_diagonal` (bool, optional): Whether the diagonal elements of the triangular matrix are assumed to be 1. Default is False.
- `overwrite_b` (bool, optional): Whether to overwrite `b`. Default is False.
- `check_finite` (bool, optional): Check for finite numbers. Default is True.

Output: The function returns the solution to the system of equations.

Usage Example:

```
import scipy.linalg as sla
import numpy as np

# Define an upper triangular matrix ‘a’ and ‘b’ vector
a = np.array([[2, 3, 4],
[0, 5, 6],
[0, 0, 7]])
b = np.array([14, 26, 21])

# Solve the triangular system
x = sla.solve_triangular(a, b)
print(x)
```

5.2 scipy.linalg.lu

Description: The `scipy.linalg.lu` function computes the LU decomposition of a matrix, which is a factorization into a lower triangular matrix and an upper triangular matrix.

Syntax:

```
scipy.linalg.lu(a, permute_l=False, overwrite_a=False,
    check_finite=True)
```

Inputs:

- `a` (array-like): The input matrix to be factored.
- `permute_l` (bool, optional): If True, permute the rows of L for the upper triangular factor to have unit diagonal elements.
- `overwrite_a` (bool, optional): If True, the function overwrites the input matrix `a`.
- `check_finite` (bool, optional): Check that the input matrix contains only finite numbers.

Output: The function returns a tuple of 3 or 2 entries:

- P is the permutation matrix.
- L is the lower triangular matrix (with unit diagonal elements if not permuted).

NOTE: If `permute_l` is `True`, returns the permuted L matrix.

- U is the upper triangular matrix.

Usage Example:

```
import scipy.linalg as la
import numpy as np

# Define a matrix 'a'
a = np.array([[2, 3, 4],
[5, 6, 7],
[8, 9, 10]])

# Compute the LU decomposition
P, L, U = sla.lu(a)

print("Permutation matrix P:")
print(P)
print("Lower triangular matrix L:")
print(L)
print("Upper triangular matrix U:")
print(U)
```

5.3 math.ceil

Description: The `math.ceil` function returns the smallest integer greater than or equal to a given number. Note that `math.floor` works similarly for greatest integer less than or equal to.

Syntax:

```
math.ceil(x)
```

Inputs:

`x` (float or int): A number.

Output:

- (int): The smallest integer greater than or equal to `x`.

Usage Example:

```
import math

# Find the smallest integer greater than or equal to 3.7
result = math.ceil(3.7)
print(result)
```